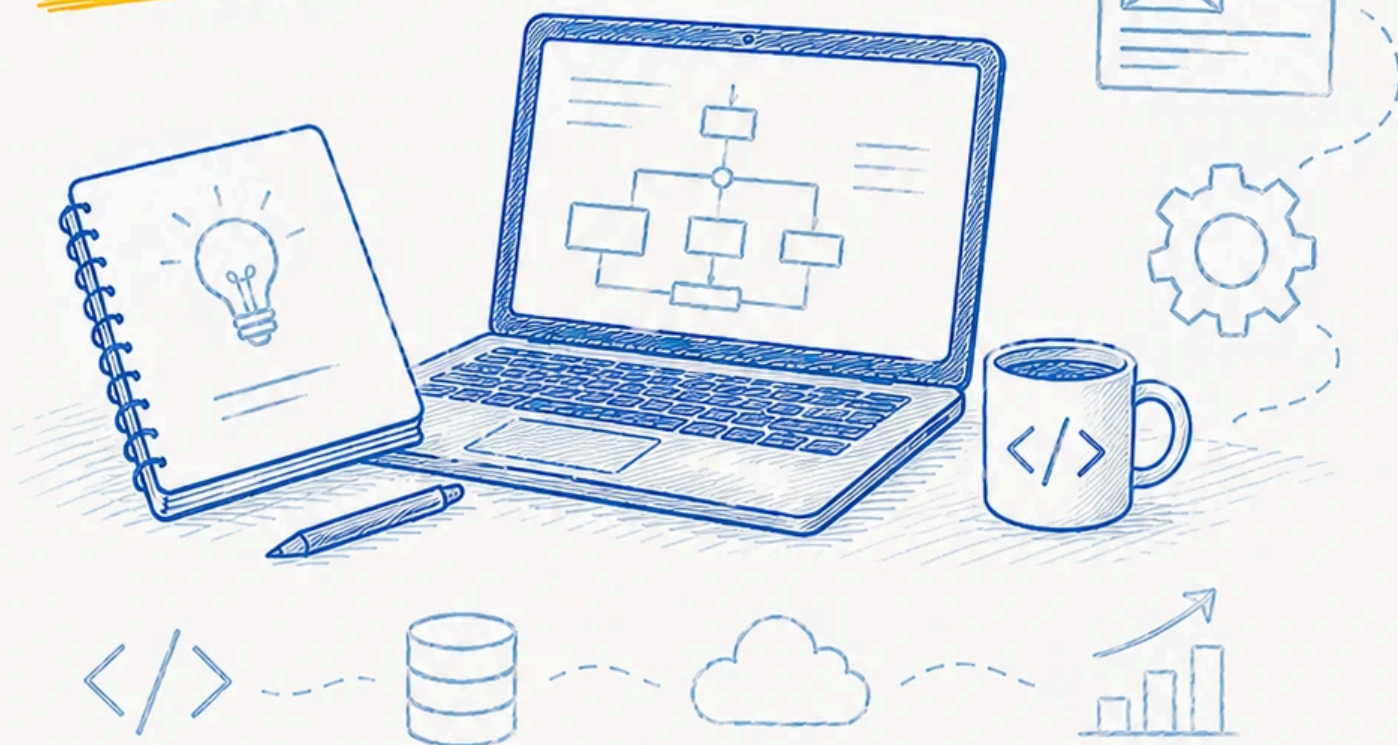


APUNTES

Proyecto intermodular de DAW

Primer y segundo curso



Por

Sergi García Barea

licencia
CC BY SA



Apuntes PI2 — Cómo evolucionar el proyecto

Sergi Garcia Barea

CC BY-SA 4.0

Apuntes PI2 — Cómo evolucionar el proyecto

Apuntes PI2 — Cómo evolucionar el proyecto

 *El día que el proyecto se hizo mayor*

Habéis llegado a PI2. El MVP de PI1 funcionaba, pero era básico. Ahora toca convertirlo en algo más serio: mejor interfaz, más funcionalidades, pruebas, seguridad, despliegue real.

El proyecto ya no es un prototipo. Es un producto que hay que hacer crecer sin que se rompa. Y eso requiere un enfoque distinto.

¿Qué cambia en PI2 respecto a PI1?

PI1 era **hacerlo funcionar**. PI2 es **hacerlo bien**.

Aspecto	PI1	PI2
Objetivo	MVP funcional	Producto completo
Frontend	HTML/CSS básico	Framework con componentes
Backend	CRUD simple	APIs, autenticación, validación
Datos	Una tabla, lo justo	Varias entidades relacionadas
Calidad	“Funciona en mi máquina”	Tests, code review
Despliegue	Local	Cloud, CI/CD
Equipo	Aprendiendo a colaborar	Flujo de trabajo establecido

Las 9 unidades de PI2

U1. Evolucionar la arquitectura

Refactorizar, pagar deuda técnica, tomar decisiones difíciles.

→

U2. Mejorar el frontend

Componentes, estado, rendimiento, experiencia de usuario.

→

U3. Integrar sistemas

APIs, comunicación entre servicios, documentación.

→

U4. Diseñar para el usuario

UX, usabilidad, accesibilidad, pruebas con usuarios.

→

U5. Asegurar la calidad

Testing, automatización, métricas de calidad.

→

U6. Proteger la aplicación

Seguridad práctica, OWASP, buenas prácticas diarias.

→

U7. Preparar el despliegue

Contenedores, entornos, configuración, empaquetado.

→

U8. Publicar y mantener

CI/CD, cloud, monitoreo, actualizaciones.

→

U9. Gestionar el proyecto

Retrospectivas, agile avanzado, mejora continua.

→

No son independientes. Muchas se solapan: no puedes desplegar sin calidad, ni mejorar el frontend sin pensar en el usuario. Úsalas según tu momento del proyecto.

🔑 U1. Evolucionar la arquitectura

📖 *El día que miraron el código de PI1 y lloraron*

“¿Esto lo escribimos nosotros?”.

Al empezar PI2, el equipo abrió el proyecto de PI1. Lo que encontraron era terrorífico: un archivo de 2000 líneas, funciones sin nombre, comentarios como “no tocar esto o explota”, y una mezcla de estilos que Dolores Umbridge consideraría cruel e inusual.

Habían pasado 3 meses. La deuda técnica acumulada en PI1 les impedía avanzar en PI2. Cada nueva funcionalidad requería descifrar código que ni ellos mismos entendían.

La deuda técnica: qué es y por qué importa

La deuda técnica es como la deuda financiera: la contraes cuando tomas atajos para ir más rápido, pero **hay que pagarla con intereses**.

Ejemplos de deuda técnica: - Copiar y pegar código en vez de crear una función reutilizable - No escribir tests “porque no hay tiempo” - Usar nombres de variables como a, tmp, cosa - Dejar comentarios como “TODO: arreglar esto”

Al principio, la deuda no se nota. Pero cuando el proyecto crece, cada cambio cuesta más caro. Llega un punto en que es más rápido **rehacer** que **modificar**.

¿Cuánta deuda tienes?

Señal	Gravedad
Archivos con más de 500 líneas	● Alta
Funciones que hacen más de una cosa	● Media
Sin tests	● Alta
Variables mal nombradas	● Baja
Código sin comentar (y no es obvio)	● Media
Importaciones circulares	● Alta

Cómo refactorizar sin romperlo todo

Paso 1: Identifica qué mejorar

No intentes arreglar todo a la vez. Prioriza:

1. Lo que más duele (cambiar una línea rompe todo)
2. Lo que más usas (el código que tocas cada semana)
3. Lo que más cuesta mantener (esa función que ya no sabes qué hace)

Paso 2: Prepara el terreno

Antes de cambiar nada, asegúrate de que puedes volver atrás: - Commit del estado actual - Crea una rama nueva para la refactorización - Si es posible, escribe tests que verifiquen el comportamiento actual

Paso 3: Cambia poco y prueba

Refactoriza en pequeñas iteraciones: 1. Extrae una función → prueba 2. Renombra una variable → prueba 3. Divide un archivo → prueba

No hagas todo a la vez o no sabrás qué rompió el cambio.

►  ¿Merece la pena refactorizar o es mejor rehacer desde cero?

Decisiones técnicas en PI2

En PI2 tomarás decisiones que no te planteaste en PI1:

¿Framework frontend o HTML plano?

Si tu interfaz es compleja, un framework (React, Vue, Svelte) te ayudará. Si son 3 pantallas, quizá no lo necesites.

¿API REST propia o usar un servicio externo?

Una API propia te da control total. Un servicio externo (Firebase, Supabase) te ahorra trabajo pero te ata a ese proveedor.

¿Base de datos relacional o NoSQL?

Si tus datos tienen relaciones claras, SQL. Si son documentos flexibles, NoSQL. ^{4 / 15} Para un proyecto DAW, SQL suele ser suficiente.

No hay respuestas universales. La respuesta correcta es **la que mejor se adapte a tu proyecto concreto**.

El proyecto que refactorizó a tiempo

Un equipo llegó a PI2 con un proyecto funcional pero desordenado. En lugar de añadir funcionalidades nuevas, dedicaron las dos primeras semanas a refactorizar:

1. Dividieron el monolito en módulos (usuarios, productos, pedidos)
2. Extrajeron la lógica de negocio de los controladores a servicios
3. Renombraron variables y funciones para que fueran legibles
4. Añadieron tests a las partes más críticas
5. Documentaron la arquitectura

Perdieron dos semanas de “avance” en nuevas funcionalidades. Pero a partir de entonces, cada nueva funcionalidad les llevaba la mitad de tiempo que antes. Terminaron PI2 con un proyecto sólido, bien estructurado y fácil de mantener.

Lección: a veces la mejor manera de avanzar es parar, ordenar la casa, y luego seguir. La deuda técnica no se paga sola.

Checklist para tu arquitectura

- ☐ He identificado la deuda técnica que arrastro de PI1
- ☐ Priorizo las refactorizaciones por impacto
- ☐ Cada cambio va acompañado de tests
- ☐ Tengo una estructura de carpetas coherente
- ☐ Las decisiones técnicas están justificadas (no “porque sí”)
- ☐ El código es legible por alguien que no lo escribió

U2. Mejorar el frontend

 *El día que el frontend se volvió complejo*

En PI1, la interfaz eran 3 páginas HTML con un poco de CSS y JavaScript básico. Funcionaba. Pero en PI2 el proyecto había crecido: ahora había 15 pantallas, formularios con validación compleja, actualizaciones en tiempo real, y datos que cambiaban según lo que hacía el usuario.

Mantener todo aquello con HTML plano era como intentar organizar una biblioteca con posts-its. Había llegado el momento de dar el salto a un framework.

¿Necesitas un framework frontend?

No todos los proyectos lo necesitan. Pregúntate:

- ¿Tienes muchas pantallas que comparten elementos?
- ¿Los datos cambian dinámicamente (sin recargar la página)?
- ¿Tienes formularios complejos con mucha validación?
- ¿El código JavaScript se te está haciendo difícil de mantener?

Si respondiste “sí” a dos o más, considera un framework.

¿Cuál elegir?

Framework	Curva	Ideal para
React	Media	Proyectos grandes, mucha comunidad
Vue	Baja	Proyectos medianos, fácil de empezar
Svelte	Baja	Proyectos donde quieres menos código
Astro	Media	Sitios con poco JavaScript, contenido estático

No elijas el que está de moda. Elige el que mejor se adapte a tu proyecto y a lo que sepas (o tengas tiempo de aprender).

►  ¿Y si no sé ningún framework?

Componentes: la pieza fundamental

Un componente es una parte reutilizable de la interfaz. Un botón, una tarjeta, un formulario, un encabezado.

Piensa en componentes desde el principio

Mira tu interfaz y pregúntate: ¿qué partes se repiten?

- Todos los botones se parecen → crea un `Button` component
- Las tarjetas de producto son iguales → crea un `ProductCard`
- El formulario de registro es similar al de login → comparte `InputField`

Cada componente debe: - Hacer una sola cosa - Recibir datos por props - No depender de nada externo

Estado: dónde vive la información

Uno de los mayores desafíos del frontend es gestionar el **estado** (los datos que cambian en la interfaz).

Regla simple: el estado vive lo más arriba posible y fluye hacia abajo.

```
App (tiene los usuarios)
├── ListaUsuarios (recibe usuarios como prop)
│   └── UsuarioCard (recibe un usuario como prop)
└── FormularioUsuario (recibe función para añadir usuario)
```

Si dos componentes necesitan los mismos datos, el estado debe estar en el componente padre de ambos.

Del HTML plano a los componentes

En PI1, un equipo hizo su interfaz con HTML plano y jQuery. Cada página era un archivo HTML independiente. El menú de navegación estaba copiado en los 5 archivos. Cuando cambiaban un enlace, tenían que actualizarlo en los 5 sitios.

En PI2 migraron a React. Crearon componentes: - NavBar (una vez, se reutiliza en todas partes) - ProductCard (se reutiliza para cada producto) - Button, Input, Modal (componentes atómicos)

El resultado: menos código, cambios más rápidos, interfaz más consistente. Lo que antes era mantenible con esfuerzo, ahora era mantenible con facilidad.

Lección: un framework no es un capricho. Cuando tu interfaz crece, es una necesidad. Pero no migres hasta que realmente lo necesites.

Rendimiento: que no tarde

Una interfaz bonita pero lenta es una mala interfaz. Cuida:

1. **Tamaño de las imágenes:** no subas fotos de 5MB, comprimelas
2. **Peticiones a la API:** no hagas 20 llamadas si puedes hacer 1
3. **Renderizado:** no renderices todo de golpe, usa paginación o carga perezosa
4. **Caché:** guarda datos que no cambian a menudo (lista de provincias, etc.)

Checklist para tu frontend

- ☐ La interfaz se ve bien en móvil, tablet y escritorio
- ☐ Los tiempos de carga son aceptables (< 3 segundos)
- ☐ Uso componentes reutilizables donde tiene sentido
- ☐ El estado está gestionado de forma coherente
- ☐ Las imágenes están optimizadas
- ☐ La navegación es intuitiva
- ☐ Los mensajes de error son claros y visibles

U3. Integrar sistemas

 *El día que el frontend y el backend no se hablaban*

“Es que el frontend espera un JSON con nombre_usuario y el backend devuelve userName”. “Pues cambia el backend”. “Pero el backend ya lo usa en 5 sitios”.

El equipo llevaba una semana intentando conectar las dos partes de su aplicación. El frontend hablaba en español, el backend en inglés. Uno esperaba arrays, el otro objetos. Los errores llegaban en formatos distintos.

Les faltaba algo que parecía burocracia pero era esencial: **definir cómo se comunican** antes de programar cada parte.

La comunicación entre partes

Tu proyecto tiene partes que necesitan comunicarse. Normalmente:

- **Frontend ↔ Backend** (API REST o GraphQL)
- **Backend ↔ Base de datos**
- **Backend ↔ Servicios externos** (email, pagos, mapas)

Cada comunicación necesita un **contrato**: qué datos se envían, qué datos se reciben, en qué formato.

Define el contrato antes de programar

Antes de que cada miembro del equipo programe su parte, acordad:

```
GET /api/usuarios → 200 { id, nombre, email }
POST /api/usuarios ← { nombre, email } → 201 { id, nombre, email }
PUT /api/usuarios/:id ← { nombre } → 200 { id, nombre, email }
DELETE /api/usuarios/:id → 204 (sin contenido)
```

Con esto, el frontend puede programar sabiendo qué va a recibir, y el backend sabiendo qué tiene que devolver. Sin sorpresas.

Herramientas para definir APIs

- **Swagger/OpenAPI:** documentación interactiva de la API
- **Postman:** para probar los endpoints
- **Insomnia:** alternativa a Postman, más ligera

► ? ¿API REST siempre o hay alternativas?

Manejo de errores en la comunicación

Los errores van a ocurrir. Lo importante es que sean **predecibles**.

En el backend

```
// Estructura de error uniforme
{
  "error": "usuario_no_encontrado",
  "mensaje": "No existe un usuario con el ID 42",
  "codigo": 404
}
```

Siempre el mismo formato, siempre el código HTTP correcto.

En el frontend

```
// Manejo de errores genérico
async function fetchAPI(url, options) {
  const res = await fetch(url, options);
  if (!res.ok) {
    const error = await res.json();
    mostrarError(error.mensaje);
    return null;
  }
  return res.json();
}
```

No manejes errores caso por caso. Crea un mecanismo genérico y reutilízalo.

Integración con servicios externos

Muchos proyectos DAW usan servicios de terceros:

- **Email:** SendGrid, Mailtrap (para pruebas)
- **Pagos:** Stripe (modo test, sin dinero real)
- **Mapas:** Google Maps, Leaflet
- **Autenticación:** Google Login, GitHub OAuth

Consejos para integrar servicios externos

1. **Lee la documentación** antes de escribir código
2. **Usa el modo test/sandbox** mientras desarrollas
3. **Maneja los errores** del servicio externo (puede caerse)
4. **No expongas claves API** en el frontend ni en el repositorio
5. **Usa variables de entorno** para las configuraciones

La integración que salvó la presentación

Un equipo había desarrollado una aplicación de reservas para restaurantes. Una de las funcionalidades que más peso tenía era el envío de emails de confirmación al hacer una reserva.

Durante el desarrollo, probaban con Mailtrap (que intercepta los emails y no los envía de verdad). Todo funcionaba.

En la presentación, conectaron el servicio real de SendGrid. Y funcionó. El tribunal recibió un email de confirmación en tiempo real. Ese detalle marcó la diferencia: demostraba que la integración era real, no simulada.

Lección: integrar un servicio externo real (aunque sea en modo test) aporta mucho valor a tu proyecto. Pero asegúrate de que funciona antes del día de la defensa.

Checklist para tus integraciones

- ☐ Frontend y backend hablan el mismo idioma (formato de datos)
- ☐ Los errores tienen un formato coherente
- ☐ La API está documentada (al menos los endpoints principales)
- ☐ Las claves y configuraciones están en variables de entorno
- ☐ Los servicios externos están en modo test/sandbox
- ☐ He probado la comunicación entre partes con casos reales
- ☐ Tengo un plan B si un servicio externo falla en la presentación

U4. Diseñar para el usuario

 *El día que probaron la app con alguien real*

“Es muy fácil de usar”, decía el equipo. Estaban tan orgullosos de su interfaz que no veían los problemas.

Hasta que un día, un compañero de otra clase probó la aplicación. “¿Dónde hago clic para empezar?”. “Esto no sé si es un enlace o un botón”. “¿Cómo vuelvo atrás?”. “He puesto mal el email y no me deja corregirlo”.

El equipo se dio cuenta de que habían diseñado la interfaz para ellos mismos, que ya sabían cómo funcionaba. No para un usuario real.

UX no es “hacerlo bonito”

UX (User Experience) es cómo se siente una persona al usar tu aplicación. No es solo el diseño visual, es:

- ¿Encuentra lo que busca?
- ¿Entiende lo que ve?
- ¿Completa las tareas sin frustrarse?
- ¿Quiere volver a usarla?

Un diseño bonito pero confuso es malo. Un diseño simple pero funcional es bueno. Un diseño bonito y funcional es excelente.

Principios básicos de usabilidad

1. Consistencia

Los botones se parecen entre sí. Los enlaces tienen el mismo color. Los formularios siguen el mismo patrón. El usuario no tiene que volver a aprender cada pantalla.

2. Feedback

Cuando el usuario hace clic, algo ocurre. Cuando una operación tarda, se muestra un indicador de carga. Cuando hay un error, se ve claramente.

3. Prevención de errores

Mejor que mostrar “contraseña incorrecta” es evitar que el usuario la escriba mal: muestra un “mostrar contraseña”, indica los requisitos antes de escribir.

4. Visibilidad del estado

El usuario sabe en todo momento dónde está y qué está pasando. “Página 3 de 10”, “Quedan 2 campos por rellenar”.

5. Libertad para deshacer

“¿Seguro que quieres eliminar esto?”. Un “deshacer” después de una acción es aún mejor.

►  ¿Cuánto tiempo debo invertir en UX?

Accesibilidad: para todos

La accesibilidad no es un extra. Es que tu aplicación pueda ser usada por **todas las personas**, incluyendo aquellas con discapacidades.

Lo mínimo que deberías hacer

1. **Contraste suficiente:** texto sobre fondo debe leerse bien
2. **Etiquetas en formularios:** cada campo debe tener su label
3. **Navegación por teclado:** debe poderse usar sin ratón (Tab, Enter)
4. **Texto alternativo en imágenes:** para lectores de pantalla
5. **Tamaño de texto:** que se pueda aumentar sin romper el diseño

Por qué importa

Porque el tribunal puede fijarse. Porque es parte de la evaluación. Porque es lo correcto. Y porque una aplicación accesible llega a más personas.

Cómo probar tu interfaz

Pruebas con usuarios (solo 3 pasos)

1. Prepara 3 tareas: “regístrate”, “busca un producto”, “haz un pedido”
2. Siéntate al lado de la persona (sin ayudarla)
3. Observa dónde duda, se equivoca, o pregunta

Lo que mirar

- ¿Cuánto tarda en completar cada tarea?
- ¿Comete errores? ¿Dónde?
- ¿Pide ayuda? ¿En qué momento?
- ¿Se frustra? ¿En qué pantalla?



Lo que descubrieron al probar con usuarios

Un equipo había desarrollado una aplicación de gestión de tareas. Estaban seguros de que era intuitiva. Hicieron la prueba con 3 personas:

Persona 1: no encontró el botón de “añadir tarea” (era gris sobre gris) **Persona 2:** intentó hacer clic en el título de la página (“creía que era un enlace”) **Persona 3:** escribió mal el email y el error solo aparecía al enviar el formulario

Los tres problemas eran fáciles de arreglar: - El botón pasó a azul con blanco - El título dejó de tener estilo de enlace - Añadieron validación en tiempo real

Una hora de pruebas con usuarios reales les ahorró semanas de quejas y bugs. Y mejoró la nota final significativamente.

Checklist para tu interfaz de usuario

- ☐ He probado la app con al menos 2 personas externas
- ☐ Los botones y enlaces se distinguen claramente
- ☐ Los errores se muestran cerca del campo correspondiente
- ☐ Hay confirmación antes de acciones destructivas
- ☐ La app se puede usar solo con teclado (Tab, Enter)
- ☐ El contraste es suficiente (herramienta: WebAIM Contrast Checker)
- ☐ Las imágenes tienen texto alternativo
- ☐ Los tiempos de carga tienen indicador visual

✅ U5. Asegurar la calidad

 El día que la app se rompió en la presentación

“Todo funciona en local”, repitió el equipo antes de la defensa. La aplicación iba perfecta en sus portátiles.

En la presentación, conectaron al proyector. Abrieron la app. La base de datos no respondía. El CSS se veía distinto. Un botón que siempre había funcionado ahora no hacía nada.

El tribunal esperaba. El equipo sudaba. “Es que en mi ordenador funciona”, dijo uno. Pero no estaban en su ordenador. Estaban en el examen.

Los tests no son para los desarrolladores. Son para los usuarios. Y el tribunal es el usuario más importante del día de la defensa.

¿Por qué hacer tests?

Los tests no son un trámite aburrido. Son un **seguro de vida** para tu proyecto. Sirven para:

- Detectar errores antes de que lleguen a producción
- Asegurarte de que los cambios no rompen lo que ya funcionaba
- Documentar cómo se espera que funcione cada parte
- Dormir tranquilo la noche antes de la presentación

Tipos de test que deberías conocer

	Tipo	Qué prueba	Ejemplo
Unitario		Una función o método aislado	“El cálculo del IVA devuelve el 21%”
Integración		Varias partes juntas	“Al crear un usuario, se guarda en BD y se devuelve 201”
Funcional (E2E)		Flujo completo del usuario	“El usuario se registra, hace login y ve su perfil”
Manual		Una persona prueba la interfaz	“Haz clic aquí, mira qué pasa”

Qué probar en tu proyecto

No hace falta probarlo todo. Prioriza:

1. **Lo que más usa el usuario** (login, registro, funcionalidad principal)
2. **Lo que puede romperse más fácilmente** (validaciones, búsquedas)
3. **Lo que más ha cambiado** (código recién escrito o refactorizado)

Nivel mínimo para aprobar

Como mínimo, deberías tener:

- Tests unitarios de las funciones core (cálculos, validaciones)
- Un test de integración del flujo principal (registro → login → acción principal)
- Prueba manual completa antes de cada entrega

►  ¿Y si no sé hacer tests?

Cómo integrar tests en tu flujo

Automatización básica

1. Escribe tests para las funcionalidades que ya tienes
2. Cuando añadas una nueva funcionalidad, escribe el test primero o al mismo tiempo
3. Antes de cada commit, ejecuta los tests
4. Si un test falla, no hagas commit hasta que lo arregles

En equipo

Cuando trabajáis en grupo, los tests evitan que el código de uno rompa el de otro. Si alguien hace un cambio que rompe un test, se entera inmediatamente, no días después.

El test que salvó la presentación

Un equipo tenía una aplicación de reservas de recursos (salas, proyectores). Dos días antes de la presentación, uno de los miembros hizo un cambio “pequeño” en el backend: “solo he cambiado el nombre de una variable”.

Ese cambio rompió la creación de reservas. No se dieron cuenta porque solo probaron manualmente el login, que seguía funcionando.

Por suerte, tenían un test automatizado para la creación de reservas. Al ejecutar los tests antes de subir los cambios, el test falló. Detectaron el error, lo corrigieron en 5 minutos, y la presentación fue un éxito.

Lección: un test automatizado te avisa cuando algo se rompe. Sin él, el error habría llegado a producción y la presentación habría sido un desastre.

Pruebas manuales: la red de seguridad

Aunque tengas tests automatizados, siempre haz una prueba manual completa antes de la defensa:

1. Abre la aplicación en un navegador **limpio** (sin datos de sesión)
2. Recorre el flujo principal del usuario paso a paso
3. Prueba casos borde: email inválido, contraseña corta, campos vacíos
4. Prueba en otro dispositivo o navegador
5. Prueba sin conexión a internet (si aplica)

Checklist para la calidad

- ☐ Las funciones críticas tienen tests unitarios
- ☐ El flujo principal tiene un test de integración
- ☐ Ejecuto los tests antes de cada commit
- ☐ He hecho una prueba manual completa antes de la entrega
- ☐ He probado la app en al menos 2 navegadores distintos
- ☐ He probado la app desde otro ordenador/dispositivo
- ☐ Los tests están en el repositorio y se ejecutan con un comando

U6. Proteger la aplicación

 *El día que alguien borró la base de datos*

“Se me ocurrió probar a poner ' ; DROP TABLE usuarios; -- en el campo de búsqueda... y funcionó”.

Un estudiante, por curiosidad, probó un ataque de inyección SQL en el proyecto de un compañero. La base de datos se borró entera. No había backup. No había vuelta atrás.

El proyecto no era público. No era un hacker malicioso. Fue un compañero probando “por si acaso”. Pero el daño fue el mismo.

La seguridad no es solo para protegerte de hackers. Es para protegerte de **cualquier error** que pueda exponer o destruir tus datos.

Seguridad no es opcional

Tu aplicación web almacena datos de usuarios: nombres, correos, contraseñas. Aunque sea un proyecto de clase, esos datos merecen protección.

Además, el tribunal valora que tu aplicación sea **segura por diseño**, no que “no tenga errores de seguridad porque no la ha visto nadie”.

Los 4 problemas más comunes (y cómo evitarlos)

1. Inyección SQL

El problema: el usuario escribe código SQL en un campo de texto y la base de datos lo ejecuta.

```
-- Si haces: SELECT * FROM usuarios WHERE email = ' + input + '
-- Y el usuario escribe: ' OR '1'=1
-- La consulta se convierte en:
SELECT * FROM usuarios WHERE email = '' OR '1'=1' -- ¡devuelve todos los usuarios!
```

Solución: usa consultas parametrizadas, nunca concatenes strings.

```
// ❌ MAL
db.query("SELECT * FROM usuarios WHERE email = '" + email + "'");
```

```
// ✅ BIEN
db.execute("SELECT * FROM usuarios WHERE email = ?", [email]);
```

2. Contraseñas en texto plano

El problema: guardas la contraseña tal como la escribe el usuario. Si alguien accede a la base de datos, tiene todas las contraseñas.

Solución: guarda siempre el **hash** de la contraseña, nunca la original.

```
// ❌ MAL
usuario.password = req.body.password;
```

```
// ✅ BIEN
```

```
const bcrypt = require('bcrypt');
usuario.password_hash = await bcrypt.hash(req.body.password, 10);
```

3. Exposición de datos sensibles

El problema: devuelves datos que no deberías (contraseñas, datos internos).

```
// ❌ MAL: devuelves la contraseña (aunque sea hash)
res.json(usuario);

// ✅ BIEN: filtras lo que devuelves
res.json({ id: usuario.id, nombre: usuario.nombre, email: usuario.email });
```

4. Falta de validación

El problema: confías en que el cliente siempre envía datos correctos.

Solución: valida **siempre** en el servidor. La validación del frontend es para mejorar la experiencia, no para proteger los datos.

► ? ¿Necesito HTTPS para un proyecto de clase?

Buenas prácticas diarias

Además de lo anterior, incorpora estos hábitos:

1. **Variables de entorno:** nunca hardcodees claves, tokens o URLs
2. **Principio de mínimo privilegio:** el usuario solo accede a lo que necesita
3. **Validación en servidor:** el frontend miente, el backend protege
4. **Logs sin datos sensibles:** no registres contraseñas ni tokens en los logs
5. **Dependencias actualizadas:** npm audit o mvn dependency-check periódicamente

El proyecto que filtró datos sin querer

Un equipo desarrolló una API para una tienda online. En el endpoint `/api/usuarios/:id` devolvían el objeto completo del usuario, incluyendo `password_hash` y `token_recuperacion`.

Durante el desarrollo no pasó nada porque solo ellos lo usaban. Pero en la presentación, el profesor pidió ver la respuesta de la API y, usando el inspector del navegador, vio que en la respuesta aparecía el hash de la contraseña.

No era un fallo crítico (el hash no se puede revertir), pero demostraba que no habían pensado en qué datos exponer. El profesor les preguntó: “¿Y si esto fuera un proyecto real? ¿Qué pasaría?”.

No suspendieron, pero perdieron puntos en el apartado de seguridad.

Lección: revisa qué datos devuelve cada endpoint de tu API. Si no es necesario que el cliente lo vea, no lo devuelvas.

Checklist para la seguridad

- ☐ Todas las consultas a BD usan parámetros, no concatenación
- ☐ Las contraseñas se guardan con hash (bcrypt)
- ☐ Los datos sensibles no se devuelven en las respuestas de la API
- ☐ El backend valida todos los datos de entrada
- ☐ Los secrets están en variables de entorno
- ☐ Uso HTTPS en el despliegue
- ☐ Los roles y permisos están implementados (al menos lo básico)
- ☐ No hay archivos de configuración con claves en el repositorio

U7. Preparar el despliegue

 *El día que ‘en mi máquina funciona’ no sirvió*

“Pues en mi ordenador va perfecto”.

El equipo llevaba una hora intentando arrancar la aplicación en el portátil del profesor. Faltaban dependencias, la versión de Node.js era distinta, la base de datos no se conectaba, las rutas de los archivos no coincidían.

Habían desarrollado durante meses en sus propios ordenadores, cada uno con su configuración, sus versiones, sus variables de entorno. Nunca habían probado a arrancar el proyecto desde cero en una máquina limpia.

“En mi máquina funciona” es la frase más peligrosa del desarrollo. Porque el tribunal no usará tu máquina.

El problema de los entornos

Tu aplicación funciona en tu ordenador porque tiene: - Tu versión de Node.js (23.x) - Tu base de datos local (MySQL 8.0) - Tus variables de entorno configuradas - Tus archivos en rutas específicas

En otro ordenador (el del profesor, un servidor) todo eso puede ser distinto. Y la aplicación se rompe.

La solución: **empaquetar** tu aplicación para que incluya todo lo que necesita.

Contenedores: la solución moderna

Un contenedor (Docker) es como una **caja** que contiene tu aplicación y todo lo que necesita para funcionar. Dentro de la caja, la aplicación ve el mismo sistema operativo, las mismas versiones, la misma configuración, independientemente de dónde se ejecute la caja.

¿Necesitas Docker?

Sí, si: - Tu proyecto tiene más de un servicio (frontend + backend + BD) - Quieres que funcione igual en cualquier máquina - Quieres desplegarlo en la nube

No, si: - Es un proyecto pequeño que solo usas tú en tu máquina - No tienes tiempo para aprender Docker

Lo mínimo que debes saber

- 1. **Dockerfile**: las instrucciones para construir tu imagen
- 2. **docker-compose.yml**: cómo se relacionan tus servicios (front, back, BD)
- 3. **.dockerignore**: qué no incluir en la imagen (node_modules, .env)

```
# Ejemplo mínimo para un backend Node.js
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

►  ¿Y si no tengo tiempo para Docker?

Más allá de Docker: buenas prácticas de empaquetado

Variables de entorno

Toda configuración que cambie entre entornos debe ir en variables de entorno: - URL de la base de datos - Claves de API - Puerto del servidor - Modo debug / producción

```
//  MAL: hardcodedo
const DB_URL = "localhost:3306/mi_db";

//  BIEN: variable de entorno
const DB_URL = process.env.DB_URL || "localhost:3306/mi_db";
```

Configuración por entorno

Ten diferentes configuraciones para desarrollo y producción:

```
config/
├─ default.js      # común a todos
├─ development.js  # base de datos local, logs detallados
└─ production.js   # base de datos remota, logs mínimos
```

El proyecto que se desplegó y funcionó

Un equipo dedicó los últimos 10 días del proyecto a preparar el despliegue: - Crearon un Dockerfile para el backend - Crearon un docker-compose.yml con backend + frontend + MySQL - Probaron en su ordenador: funcionaba - Subieron todo a un VPS (servidor en la nube): funcionaba - El día de la presentación, abrieron la URL en el navegador del tribunal: funcionaba

No pasaron horas arreglando dependencias ni configuraciones. No hubo “en mi máquina funciona”. Hubo “funciona en todas partes”.

Lección: el despliegue no es un paso opcional al final. Es parte del proyecto. Dedicarle tiempo es tan importante como programar. Un proyecto desplegado y accesible desde internet causa mucha mejor impresión que uno que solo funciona en un portátil.

Checklist para tu despliegue

- ☐ He probado a arrancar el proyecto desde cero en una máquina limpia
- ☐ Las configuraciones están en variables de entorno
- ☐ Tengo un README con instrucciones de instalación claras
- ☐ Si uso Docker, el docker-compose.yml funciona en otra máquina
- ☐ Los puertos y rutas están configurados (no hardcodedos)
- ☐ Las dependencias están especificadas con versiones exactas
- ☐ He probado el proyecto en un navegador que no sea el mío

U8. Publicar y mantener

 *El día que la aplicación salió al mundo*

“Ya está. Hemos terminado”, dijo el equipo. Pero el profesor preguntó: “¿Y dónde puedo verla funcionando?”.

Silencio. La aplicación estaba en sus portátiles. En localhost.

Desplegar no es “lo que haces al final si sobra tiempo”. Es parte del proyecto. Y deberías empezar a pensarlo **antes** de la última semana.

¿Por qué desplegar?

Un proyecto que solo funciona en localhost está **incompleto**. Desplegarlo demuestra que:

- La aplicación funciona fuera de tu entorno controlado
- Sabes configurar un servidor
- Entiendes el ciclo de vida de una aplicación web
- El tribunal puede probarlo sin instalar nada

Opciones de despliegue gratuito

Plataforma	Ideal para	HTTPS
Vercel	Frontend (React, Vue, Astro)	✓
Netlify	Frontend estático	✓
Render	Backend + BD gratuita (limitado)	✓
Railway	Backend + BD	✓
GitHub Pages	Sitios estáticos	✓
Firebase	Apps con backend ligero	✓
PythonAnywhere	Backend Python	✓

Elige la que mejor se adapte a tu stack. La mayoría tiene planes gratuitos suficientes para un proyecto DAW.

►  ¿Y si el despliegue cuesta dinero?

CI/CD: que se despliegue solo

CI/CD significa **Integración Continua / Despliegue Continuo**. En cristiano: cada vez que subes código al repositorio, los tests se ejecutan automáticamente y, si todo va bien, la aplicación se despliega sola.

Lo mínimo que deberías tener

Un flujo CI/CD básico con GitHub Actions:

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test          # tests automáticos
      - run: npm run build     # construir
      - run: npm run deploy    # desplegar
```

Con esto, cada vez que haces `git push` a `main`, el proyecto se prueba y se despliega automáticamente. Sin intervención manual.

Ventajas de CI/CD

- **Olvidate de desplegar manualmente:** un `git push` lo hace todo
- **Los tests se ejecutan siempre:** si algo falla, no se despliega
- **El tribunal ve la última versión:** siempre que haces un push
- **Menos errores humanos:** no olvidas un paso del despliegue

El despliegue que impresionó al tribunal

Un equipo configuró CI/CD desde el principio. Cada vez que hacían un push a `main`, el proyecto se desplegaba automáticamente en Render.

El día de la presentación: 1. Abrieron la URL en el navegador del tribunal 2. La aplicación funcionaba 3. El profesor pidió ver el código 4. Enseñaron el repositorio con el GitHub Actions configurado 5. El profesor hizo un cambio menor en una página, hicieron push, y en 2 minutos el cambio estaba visible en producción

El tribunal quedó impresionado. No solo habían hecho una aplicación, habían montado un **proceso de desarrollo profesional**.

Lección: CI/CD no es solo para empresas. Es una práctica que demuestra que entiendes cómo funciona el desarrollo de software real. Y en un proyecto DAW, marca la diferencia.

Mantenimiento post-despliegue

Una vez desplegada, la aplicación necesita mantenimiento mínimo:

1. **Monitoreo básico:** ¿está funcionando? ¿responde? (herramientas: UptimeRobot)
2. **Logs:** ¿hay errores? revisalos periódicamente
3. **Actualizaciones:** las dependencias se quedan obsoletas, actualiza lo crítico

Para un proyecto DAW, el mantenimiento se reduce a “comprobar que funciona antes de la presentación”. Pero saber que existe es parte de la formación.

Checklist para tu despliegue

- ☐ He elegido una plataforma de despliegue gratuita
- ☐ He configurado HTTPS
- ☐ Las variables de entorno están configuradas en la plataforma
- ☐ He probado la aplicación desplegada (no solo en local)
- ☐ Tengo CI/CD configurado (GitHub Actions u otro)
- ☐ Los tests se ejecutan automáticamente antes de desplegar
- ☐ La URL de la aplicación está en la memoria del proyecto
- ☐ He probado la app desplegada desde otro dispositivo

U9. Gestionar el proyecto

 *El día que miraron atrás y vieron todo lo aprendido*

Última semana de PI2. El equipo miraba el proyecto terminado. Recordaban cómo empezaron: con una idea difusa, sin saber por dónde tirar, con miedo a no llegar.

Ahora tenían una aplicación funcionando, desplegada, con tests, documentada. No era perfecta, pero era suya. Y habían aprendido más que en ninguna otra asignatura.

— “¿Qué haríais diferente si empezaraís de nuevo?”, preguntó el profesor.

Y ahí empezó la retrospectiva más importante del proyecto.

La retrospectiva: aprende de tu experiencia

Una retrospectiva es una reunión donde el equipo reflexiona sobre lo que ha funcionado y lo que no. No es opcional: es cómo mejoras para la próxima vez.

Cómo hacer una retrospectiva

Tres preguntas. Todos responden:

1. **¿Qué salió bien?** (para repetirlo)
2. **¿Qué salió mal?** (para evitarlo)
3. **¿Qué haríamos diferente?** (para mejorar)

No vale “todo bien” ni “todo mal”. Hay que ser específico.

Qué salió bien	Qué salió mal	Qué haríamos diferente
Nos coordinamos bien al principio	Dejamos la documentación para el final	Documentar desde el día 1
El diseño fue sólido	Subestimamos los tiempos	Multiplicar estimaciones por 2
La interfaz quedó profesional	No hicimos suficientes tests	Escribir tests antes del código

La retrospectiva no es un trámite

Si solo haces la retrospectiva porque el profesor la pide, pierdes su valor. La retrospectiva es para **ti**. Para que en tu próximo proyecto (trabajo de fin de ciclo, proyecto profesional) no cometas los mismos errores.

►  ¿Y si mi proyecto ha ido mal? ¿Merece la pena hacer retrospectiva?

Decisiones difíciles en la recta final

En PI2 te enfrentarás a decisiones que en PI1 no surgieron:

“**¿Recortamos funcionalidades o trabajamos fines de semana?**”

Siempre recorta funcionalidades. Tu salud y tu cordura valen más que un par de features extra. El tribunal prefiere un proyecto completo con menos funcionalidades que uno lleno de bugs.

“**¿Refactorizamos o añadimos lo que falta?**”

Refactoriza solo si el código actual te impide avanzar. Si funciona y es mantenible, déjalo. Prioriza las funcionalidades que el tribunal va a evaluar.

“**¿Presentamos el proyecto aunque no esté perfecto?**”

Siempre. Un proyecto imperfecto pero presentado vale más que un proyecto perfecto pero no entregado. Y nunca sabes qué van a valorar del tribunal.

La gestión del equipo en PI2

En PI2 el equipo ya se conoce. Ya sabe quién trabaja, quién no, quién se estresa, quién tira del carro. La clave ahora es **no perder la motivación**:

- Celebrad los hitos pequeños (primer test que pasa, primera pantalla)
- Repartid el trabajo según las fortalezas de cada uno
- Si alguien está sobrecargado, redistribuid

- Si alguien no rinde, habladlo pronto y claro

La retrospectiva que cambió la forma de trabajar

Un equipo terminó PI1 con la sensación de que había ido regular. No mal, pero tampoco bien. En la retrospectiva identificaron:

Qué falló: - No se reunían con regularidad - Cada uno iba por libre - Dejaron la documentación para el final - Un miembro hizo mucho menos que los demás

Qué harían diferente en PI2: - Reuniones cada 3 días, 15 minutos, sin falta - Tablero Kanban compartido (GitHub Projects) - Documentar a la vez que programaban - Reparto explícito de tareas desde el principio

En PI2 aplicaron todo. No fue perfecto, pero fue mejor. Mucho mejor. Terminaron con un proyecto más sólido y un equipo más unido.

Lección: un proyecto no es solo el código. Es el equipo, la organización, la comunicación. Y eso se entrena. La retrospectiva es la herramienta para entrenarlo.

Checklist para el final del proyecto

- ☐ He hecho una retrospectiva con el equipo
- ☐ Identifico lo que haría diferente la próxima vez
- ☐ El proyecto está completo (no “casi” sino funcional)
- ☐ He probado la aplicación de principio a fin
- ☐ La memoria y la presentación están listas
- ☐ He ensayado la presentación al menos una vez
- ☐ Sé responder preguntas sobre decisiones técnicas
- ☐ Estoy orgulloso del trabajo hecho (aunque no sea perfecto)